Study Tasks — Local-First Todo Application

COMS3011A Lab 1 — Supporting Documentation

AI Declaration

This project was developed with the assistance of AI tools. Claude (Anthropic) was used throughout; ChatGPT was used throughout the development process as a programming assistant. It was primarily used to explain React, Next.js, SQLite, and software architecture concepts, review code, suggest improvements, and assist with debugging.

What AI was used for

Explanation and instruction. React and Next.js concepts were explained on request.

Code generation, incrementally. Each feature was requested and reviewed in turn — the create form, SQLite persistence, the edit dialog, archiving, sorting, the overdue rule, and the test suite. The AI did not generate the application in a single pass; each feature was integrated and tested before the next was started.

Debugging. Errors were diagnosed by pasting terminal output. These included a `ReferenceError` traced to a comma typed in place of a dot, a `SqliteError: no such column` caused by `CREATE TABLE IF NOT EXISTS` silently skipping a schema change, a `node-gyp` build failure that led to pinning `better-sqlite3` to v11, and a Windows file-lock error in the test teardown that required closing the database handle before deleting the file.

Direction and correction by the author

- **Validation rules were specified.** AI initially proposed making only the title mandatory, but then I decided that title, due date and topic are required and description optional, and the validation in both the form and the API routes was changed to match.
- AI placed the archive button inside the collapsed dropdown body, but then I requested it beside the title, which meant handling the click inside `<summary>` with `stopPropagation` and `preventDefault` so it would not toggle the panel. The sort control and create button were likewise moved onto a single row.
- When the AI supplied code containing functions belonging to a later feature, I asked whether they could be omitted so each commit would represent one working feature. `archiveTask` was consequently held back until the archiving commit.

Third-Party Code

Next.js — Provides the App Router, which gives both the React UI and the HTTP API routes in one project. Chosen because the brief specifies it, and because its route handlers let the SQLite access stay server-side while the interactive UI runs in the browser.

React — The component and state model behind the UI. Used for the task list, the create/edit dialog, and the controlled form inputs.

better-sqlite3 — The SQLite driver. Chosen over the async alternatives because its synchronous API suits a single-user local application, where there is no concurrency to manage and no benefit to callbacks or promises around every query. Pinned to v11: v12 has no prebuilt binary for Node 22 and falls back to compiling from source, which fails on machines without Visual Studio build tools.

ESLint — Catches common mistakes and keeps the code consistent. Included by default with `create-next-app` and kept rather than removed.

No testing library was added. Node 22 includes a test runner (`node --test`) and an assertion module, which was sufficient here and avoids a dependency.

Database Design

All data lives in a single SQLite database, `tasks.db`, created on first run by `lib/db.js`.

Table: tasks

COLUMN	TYPE	CONSTRAINTS	PURPOSE
`id`	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique identifier, assigned by SQLite
`title`	TEXT	NOT NULL	Task title
`description`	TEXT	NOT NULL	Optional detail; stored as an empty string when not supplied
`dueDate`	TEXT	NOT NULL	ISO date, e.g. 2026-08-10
`topic`	TEXT	NOT NULL	Subject the task belongs to
`status`	TEXT	NOT NULL DEFAULT 'todo', CHECK IN ('todo', 'in-progress', 'complete')	Workflow state
`archived`	INTEGER	NOT NULL DEFAULT 0	Flag: 0 active, 1 archived

Relationships

There is one table and no foreign keys. Topic is a free-text field on the task rather than a separate entity, because the brief describes it as information the task carries rather than something the user manages independently — there is no requirement to rename a topic across tasks, list topics that have no tasks, or attach anything else to one. A `topics` table would add a join without adding capability.

Design decisions

Statuses are constrained in the schema, not just the UI. The `CHECK` constraint means the three values are fixed at the data layer, so no request reaching the API — however it is formed — can introduce a fourth. The same list lives in `lib/constants.js` and is imported by the schema, the API validation and the radio buttons, so the three cannot drift apart.

Archive is a flag on the task, not a move. Archiving sets `archived = 1`; the row is never copied to another table and never removed. `lib/db.js` contains no `DELETE` statement at all. Archived tasks are read back with every query and displayed in a separate list, where they can be restored.

Overdue is derived, never stored. There is no overdue column and no overdue status. `lib/overdue.js` computes it at read time from the due date and status: a task is overdue if its due date is in the past, it is not complete, and it is not archived. Storing it would go stale overnight, since a task becomes overdue through the passage of time rather than through any write.

Dates are ISO strings. SQLite has no date type. The `YYYY-MM-DD` format sorts correctly as text, which means `ORDER BY dueDate` and the overdue comparison both work on plain string ordering, and it is what `<input type="date">` produces natively.

Sorting happens in SQL. `getTasks` takes a sort key mapped through a fixed lookup object to an `ORDER BY` clause. Column names cannot be bound as query parameters, so the lookup restricts the clause to three known strings rather than interpolating anything from the request. Sorting by status uses a `CASE` expression to order todo → in-progress → complete, rather than the alphabetical order which would put complete first.

Project Structure

```
app/
  api/
    tasks/
      route.js        GET (list, sortable) and POST (create)
      [id]/
        route.js      PATCH (edit, and archive toggle)
  page.js             the UI
  page.module.css
lib/
  db.js               schema and queries
  constants.js        the three statuses, shared by client and server
  overdue.js          the overdue rule, kept pure so it can be tested
tests/
  db.test.js          creation, archiving, status constraint
  overdue.test.js     the overdue rule
```

AI Declaration: The preceding document was reviewed and edited with: Claude-Web[Claude Opus 5]